Asincronía y funciones callbacks en JavaScript

Objetivo: Presentar el tema de la clase y la ruta de trabajo.

Pregunta para abrir: ¿por qué una app sigue respondiendo mientras descarga una foto?

Ruta de la clase:

1. Síncrono vs. asíncrono
2. Un solo hilo, Event Loop y código no bloqueante
3. Funciones callbacks y Callback Hell
4. Vistazo a promesas y async/await
5. Práctica: checkpoints → guiados → casa → reto



¿Qué es lo síncrono?

Objetivo: Reconocer el modelo de ejecución con el que aprendimos a programar.

Síncrono = las sentencias se ejecutan **en orden, una detrás de otra**, sin pausas.

Según la RAE, *síncrono* significa que coincide en el tiempo.

Una línea no empieza hasta que la anterior terminó.

```
let x = 1;
x++;
console.log(x);
```

Consola:

```
2
```

¿Qué es la asincronía?

Objetivo: Definir la asincronía y entender por qué la necesitamos.

Asíncrono = las sentencias aparecen en un orden, pero **se ejecutan en otro**: el programa no espera a que termine una tarea lenta para seguir.

Tareas lentas típicas: leer o escribir archivos, pedir datos a un servidor, consultar una base de datos.

Ventajas: **eficiencia** (no se queda quieto), **mejor experiencia de usuario** (la app no se congela), **varias tareas en curso** a la vez.

```
console.log("Pido el café");
setTimeout(function () {
  console.log("Pasaron 2 segundos: café listo");
}, 2000);
console.log("Mientras tanto, reviso el celular");
```

Consola:

```
Pido el café
Mientras tanto, reviso el celular
Pasaron 2 segundos: café listo ← 2 s después
```

setTimeout en detalle

Objetivo: Usar setTimeout correctamente y predecir el orden de varios temporizadores.

Sintaxis: setTimeout(función, milisegundos) → 1000 ms = 1 segundo.

Los temporizadores salen **por tiempo de espera**, no por la línea donde están escritos.

Se pasa **la función** (avisar), no su resultado (avisar()): con paréntesis se ejecuta de inmediato.

```
setTimeout(function () {
  console.log("Postre (3 s)");
}, 3000);
setTimeout(function () {
  console.log("Entrada (1 s)");
}, 1000);
console.log("Pedido enviado a la cocina");
```

Consola:

```
Pedido enviado a la cocina
Entrada (1 s)
Postre (3 s)
```

🤔 Pregunta rápida

Objetivo: Comprobar la diferencia entre síncrono y asíncrono.

Pregunta 1

¿Qué imprime este código y en qué orden?

```
console.log("A");
setTimeout(function () {
  console.log("B");
}, 1000);
console.log("C");
```

Pregunta 2

El número que recibe setTimeout, ¿está en segundos o en milisegundos?

Pregunta 3

¿Por qué la asincronía mejora la experiencia del usuario?

Un solo hilo y el Event Loop

Objetivo: Entender cómo JavaScript atiende tareas lentas con un único hilo.

JavaScript tiene **un único hilo**: ejecuta una sola instrucción a la vez.

Lo que compensa esa limitación es su gestión asíncrona:

1. Una función como setTimeout entrega la espera al entorno (navegador o Node.js).
2. El callback queda guardado.
3. JavaScript sigue con el resto del código (**pila de llamadas**).
4. Cuando la espera termina, el callback pasa a una **cola**.
5. El **Event Loop** lo lleva a la pila **solo cuando la pila está vacía**.

Por eso un setTimeout de 0 ms igual sale después del código síncrono.

```
console.log("1. En la pila");
setTimeout(function () {
  console.log("3. Llegó desde la cola");
}, 0);
console.log("2. En la pila");
```

Consola:

```
1. En la pila
2. En la pila
3. Llegó desde la cola
```

JavaScript: lenguaje no bloqueante

Objetivo: Explicar qué significa "no bloqueante" y cómo cambia según el entorno.

No bloqueante: una tarea en curso **no detiene** el resto del programa; la espera ocurre en segundo plano.

El lenguaje ejecuta su código de forma síncrona; lo que lo hace no bloqueante son las **herramientas asíncronas del entorno**.

Node.js

Pensado desde el inicio para no bloquear: entrada/salida asíncrona.
Ideal para servidores y apps de red.

Navegador web

También ofrece herramientas asíncronas (temporizadores, red, clics).
Si se ejecuta código síncrono muy pesado, la página se congela.

En la siguiente diapositiva lo vemos en código.

Código: proceso no bloqueante

Objetivo: Seguir paso a paso un proceso que no bloquea el hilo principal.

Columna 1 — Explicación

Simula una tarea que tarda 2 s, como una petición a un servidor.

Los números en los comentarios indican el orden real en la consola.

"Fin del programa." sale **antes** que "completada", porque setTimeout no bloquea.

El callback recibido se ejecuta al final, cuando la espera termina.

Columna 2 — Código

```
function tareaNoBloqueante(callback) {
 console.log("Iniciando tarea no bloqueante..."); // 2
 setTimeout(function () {
 console.log("Tarea no bloqueante completada."); // 5
 callback();
 }, 2000);
 console.log("Tarea después del setTimeout..."); // 3
}
console.log("Inicio del programa."); // 1
tareaNoBloqueante(function () {
 console.log("Continuando con el resto de tareas."); // 6
});
console.log("Fin del programa."); // 4
```

🤔 Pregunta rápida

Objetivo: Verificar la comprensión del Event Loop y del código no bloqueante.

Pregunta 1

¿Qué imprime este código?

```
setTimeout(function () {
  console.log("1");
}, 0);
console.log("2");
setTimeout(function () {
  console.log("3");
}, 0);
```

Pregunta 2

¿JavaScript puede ejecutar dos funciones exactamente al mismo tiempo?

Pregunta 3

En tareaNoBloqueante, ¿por qué "Fin del programa." aparece antes que "Tarea no bloqueante completada."?

Cuando el hilo sí se bloquea

Objetivo: Reconocer que el código síncrono pesado bloquea y retrasa los temporizadores.

Los milisegundos de setTimeout son un **tiempo mínimo**, no exacto.

Si un bucle largo ocupa la pila, ningún callback puede entrar hasta que termine.

En el navegador, eso se nota como una página "congelada".

el único hilo de JavaScript queda ocupado todo ese tiempo, más o menos medio segundo en un equipo normal. **Los milisegundos de** `setTimeout` **son un tiempo mínimo, no exacto.** Pedimos 0 ms y el mensaje salió unos 500 ms después, porque el hilo estaba ocupado.

```
setTimeout(function () {
  console.log("Temporizador de 0 ms");
}, 0);
let i = 0;
while (i < 1000000000) {
  i++;
} // el hilo queda ocupado
console.log("Bucle terminado");
```

Consola:

```
Bucle terminado
Temporizador de 0 ms
```

Tareas asíncronas comunes

Objetivo: Identificar operaciones asíncronas del día a día (solo para reconocerlas).

Tarea	Ejemplo mínimo (navegador)	Qué pasa
Esperar un tiempo	setTimeout(fn, 1000)	Ejecuta fn en el futuro
Pedir datos a una URL	fetch("datos.json")	Descarga sin congelar la página
Reproducir un .mp3	new Audio("alerta.mp3").play()	El sonido sigue mientras el código avanza
Voz del navegador	speechSynthesis.speak(new SpeechSynthesisUtterance("Hola"))	Lee en voz alta sin detener el programa
Sensor del celular (GPS)	navigator.geolocation.getCurrentPosition(function (pos) { ... })	Recibe la ubicación en un callback

En la práctica de hoy solo usaremos **setTimeout** para simular estas esperas.

¿Qué son las funciones callbacks?

Objetivo: Definir un callback y usarlo como argumento.

Callback = **función que se pasa como argumento** a otra función, para que esta la ejecute cuando corresponda.

Permiten reutilizar funciones de forma flexible.

Un callback **no siempre es asíncrono**: forEach ejecuta su callback de inmediato; setTimeout lo ejecuta después.

```
function saludar(nombre, despedida) {
  console.log(`Hola, ${nombre}`);
  despedida();
}
saludar("Ana", function () {
  console.log("¡Hasta luego!");
});
```

Consola:

```
Hola, Ana
¡Hasta luego!
```

🤔 Pregunta rápida

Objetivo: Repasar bloqueo del hilo y callbacks síncronos.

Pregunta 1

¿Qué imprime este código y en qué orden?

```
const notas = [3, 4];
console.log("Antes");
notas.forEach(function (n) {
  console.log(n + 1);
});
console.log("Después");
```

Pregunta 2

¿Qué diferencia hay entre saludar("Ana", despedir) y saludar("Ana", despedir())?

Pregunta 3

Si un bucle tarda 3 s y hay un setTimeout de 1 s, ¿cuándo se ejecuta el temporizador?

Código: operaciones matemáticas con callbacks

Objetivo: Ver cómo una misma función se comporta distinto según el callback que recibe.

Columna 1 — Explicación

realizarOperacion recibe dos números y un callback.

sumar y multiplicar son dos callbacks con tareas distintas.

Se llama dos veces a la misma función, cada vez con un callback diferente.

Todo es síncrono: no hay esperas.

Columna 2 — Código

```
function realizarOperacion(num1, num2, operacionCallback) {
  console.log(`Operación: ${num1} y ${num2}`);
  operacionCallback(num1, num2);
}
function sumar(a, b) {
  console.log(`Suma: ${a + b}`);
}
function multiplicar(a, b) {
  console.log(`Multiplicación: ${a * b}`);
}
realizarOperacion(5, 3, sumar);
realizarOperacion(5, 3, multiplicar);
```

Consola:

```
Operación: 5 y 3
Suma: 8
Operación: 5 y 3
Multiplicación: 15
```

Entregar resultados con callbacks

Objetivo: Recibir el resultado de una tarea asíncrona y manejar errores con un callback.

Un return dentro de setTimeout **no** llega a quien llamó la función: cuando se ejecuta, esa función ya terminó.

Solución: entregar el dato llamando a un callback.

Patrón común: callback(error, resultado) → si no hay error, se envía null en el primer lugar.

```
function consultarSaldo(callback) {
  setTimeout(function () {
    callback(250000);
  }, 1000);
}
consultarSaldo(function (saldo) {
  console.log(`Saldo: $${saldo}`);
});
console.log("Consultando...");
```

Consola:

```
Consultando...
Saldo: $250000 ← 1 s después
```

Callback Hell (infierno de callbacks)

Objetivo: Reconocer la "pirámide de la perdición" y por qué complica el código.

Columna 1 — Explicación

Ocurre al encadenar muchas tareas asíncronas: cada paso espera al anterior, y cada callback queda dentro de otro.

Se forma una pirámide (*Pyramid of Doom*): callbacks claros → encadenados → anidamiento profundo → infierno.

Problemas: difícil de leer, de modificar y de encontrar errores.

Columna 2 — Código

```
function paso(mensaje, siguiente) {
  setTimeout(function () {
    console.log(mensaje);
    siguiente();
  }, 500);
}
paso("Tomar pedido", function () {
  paso("Cocinar", function () {
    paso("Empacar", function () {
      paso("Entregar", function () {
        console.log("Fin");
      });
    });
  });
});
```

Consola (cada 0,5 s): Tomar pedido → Cocinar → Empacar → Entregar → Fin

Aplanar la pirámide con funciones con nombre

Objetivo: Reescribir un Callback Hell como una cadena legible de funciones.

Cada paso se convierte en una función con nombre que indica cuál sigue (diapositiva anterior).

El resultado es el mismo, pero el código se lee de arriba abajo.

```
function tomar() {
  paso("Tomar pedido", cocinar);
}
function cocinar() {
  paso("Cocinar", empacar);
}
function empacar() {
  paso("Empacar", entregar);
}
function entregar() {
  paso("Entregar", function () {
    console.log("Fin");
  });
}
tomar();
```

Consola (cada 0,5 s):

```
Tomar pedido
Cocinar
Empacar
Entregar
Fin
```

Promesas (vistazo)

Objetivo: Conocer el mecanismo moderno que reemplaza a los callbacks anidados.

Una promesa representa un resultado que **llegará en el futuro**: se cumple (resolve) o falla (reject).

.then recibe el éxito y .catch el error; las operaciones se pueden encadenar sin pirámide.

```
const pedido = new Promise(function (resolve, reject) {
  setTimeout(function () {
    resolve("Pizza lista");
  }, 1000);
});
pedido.then(function (r) {
  console.log(r);
})
.catch(function (e) {
  console.log(`Error: ${e}`);
});
```

Consola:

```
Pizza lista ← 1 s después
```

```
const hayQueso = true;
const pedido = new Promise(function (resolve, reject) {
  setTimeout(function () {
    if (hayQueso === true) {
      resolve("Pizza lista");
    } else {
      reject("No hay queso");
    }
  }, 1000);
});
pedido
  .then(function (r) { console.log(r); })
  .catch(function (e) { console.log(`Error: ${e}`); });
```

async/await y ¿cuál mecanismo elegir?

Objetivo: Conocer async/await y comparar las tres formas de manejar la asincronía.

async marca una función; dentro, await espera una promesa con apariencia de código síncrono.

```
async function esperarPedido() {
  const r = await pedido; // la promesa de la diapositiva anterior
  console.log(r);
}
esperarPedido();
```

Consola: Pizza lista (1 s después)

Mecanismo	Cuándo usarlo
Callbacks	Operaciones simples o APIs antiguas. Cuidado con el Callback Hell.
Promesas	Operaciones encadenadas y manejo de errores estructurado.
async/await	Código más limpio; recomendado en la mayoría de casos modernos.

Hoy practicamos con callbacks; promesas y async/await se profundizan más adelante.

🤔 Pregunta rápida

Objetivo: Repasar el Callback Hell, las promesas y async/await.

Pregunta 1

¿Qué imprime este código?

```
const p = new Promise(function
(resolve) {
  resolve("listo");
});
p.then(function (v) {
  console.log(v);
});
console.log("primero");
```

Pregunta 2

¿Qué palabra debe tener una función para poder usar await dentro?

Pregunta 3

Para encadenar cinco pasos de un pedido, ¿qué mecanismo elegirías y por qué?

Chuleta: ¿qué necesito y qué uso?

Objetivo: Relacionar problemas típicos con la herramienta de JavaScript que los resuelve.

Necesito...	**Uso...**
Ejecutar algo después de un tiempo	setTimeout(funcion, ms)
Que una función haga distintas cosas según el caso	Pasarle un **callback**
Recibir el resultado de una tarea que tarda	Un callback que reciba el dato (callback(dato))
Avisar si algo salió mal	callback(error, resultado) y un if (error !== null)
Encadenar pasos asíncronos sin pirámide	Funciones con nombre (o promesas)
Que un mensaje salga antes que la tarea lenta	Escribirlo como código síncrono después de iniciarla
Sumar un dato que llega como texto	Number(valor) antes de operar
Comparar sin sorpresas de tipo	=== y !==

Práctica: orden de trabajo y reglas de juego

Objetivo: Conocer cómo vamos a trabajar y las reglas para todos los ejercicios.

Orden: ✅ Checkpoints → 🏫 Guiados → 🏠 Para la casa → 🔥 Reto integrador.

Reglas:

- Usa index.html + js/main.js, con el <script> justo antes de </body>.
- No modifiques los datos ni el código base, salvo donde el ejercicio lo indique.
- Comenta cada requisito: // R1, // R2...
- const por defecto; let solo si hay que reasignar; nunca var.
- Compara con === y !==.
- Sin librerías externas.
- Verifica cada resultado en la consola (F12).

✅ Checkpoint 1 · Cafetería: ¿quién sale primero?

Objetivo: Combinar código síncrono, setTimeout y Event Loop para predecir un orden.

Columna 1 — Retos

1. Antes de ejecutar, escribe en un comentario el orden en que crees que saldrán los mensajes. Luego compruébalo.
2. **Explica por qué** "agua" sale después del cajero aunque espera 0 ms.
3. Agrega el pedido "Listo: aromática" para que salga entre el tinto y el capuchino, sin mover ninguna línea.
4. **Explica por qué** el orden de salida no es el orden de escritura.

Columna 2 — Código base

```
console.log("Abre la cafetería");
setTimeout(function () {
 console.log("Listo: capuchino");
}, 3000);
setTimeout(function () {
 console.log("Listo: tinto");
}, 1000);
setTimeout(function () {
 console.log("Listo: agua");
}, 0);
console.log("El cajero atiende al siguiente cliente");
```

✅ Checkpoint 2 · Calculadora de propinas

Objetivo: Combinar callbacks, conversión de tipos y el error de pasar fn() en lugar de fn.

Columna 1 — Retos

1. Ejecuta y **explica por qué** el total es $500005000.
2. Corrígelo dentro de calcularTotal, sin cambiar la constante cuenta.
3. Crea un callback mostrarDividido que muestre cuánto paga cada una de 4 personas. No modifiques calcularTotal.
4. **Explica qué pasa** si llamas calcularTotal(cuenta, 10, mostrarEnPesos()).

Columna 2 — Código base

```
const cuenta = "50000"; // llega como texto
function calcularTotal(valor, porcentaje, mostrar) {
 const propina = valor * porcentaje / 100;
 const total = valor + propina;
 mostrar(total);
}
function mostrarEnPesos(monto) {
 console.log(`Total a pagar: $${monto}`);
}
calcularTotal(cuenta, 10, mostrarEnPesos);
```

✅ Checkpoint 3 · Buscador de la biblioteca

Objetivo: Combinar setTimeout, entrega de resultados con callback y comparación estricta de tipos.

Columna 1 — Retos

1. ¿Qué imprime el código? **Explica por qué**.
2. Reescribe buscarLibro para que reciba un callback y entregue el libro con él.
3. Buscando "1" no aparece nada. **Explica por qué** y corrígelo sin cambiar el ===.
4. Si el libro no existe, el callback debe recibir null y se debe imprimir "No encontrado". Prueba con el id 7.

Columna 2 — Código base

```
const libros = [
 { id: 1, titulo: "Cien años de soledad", disponible: true },
 { id: 2, titulo: "La vorágine", disponible: false },
];
function buscarLibro(id) {
 setTimeout(function () {
 for (const libro of libros) {
 if (libro.id === id) {
 return libro;
 }
 }
 }, 1000);
}
const resultado = buscarLibro("1"); // viene de la caja de
búsqueda
console.log(resultado);
```

✅ Checkpoint 4 · Seguimiento de un domicilio

Objetivo: Combinar Callback Hell, alcance de variables y orden de ejecución.

Columna 1 — Retos

1. Ejecuta y **explica por qué** aparece ReferenceError en la última línea. Corrígelo.
2. ¿Cuántos niveles de anidamiento tiene la pirámide?
3. Aplánala con funciones con nombre (recibido, preparando, enCamino, entregado).
4. **Explica por qué** "Repartidor asignado" sale antes que "Pedido recibido".

Columna 2 — Código base

```
function paso(mensaje, siguiente) {
 setTimeout(function () {
 console.log(mensaje);
 siguiente();
 }, 500);
}
paso("Pedido recibido", function () {
 const repartidor = "Ana";
 paso("Preparando pedido", function () {
 paso(`En camino con ${repartidor}`, function () {
 paso("Entregado", function () {
 console.log("Fin del seguimiento");
 });
 });
 });
});
console.log(`Repartidor asignado: ${repartidor}`);
```

🏫 Guiado 1 · Tienda en línea: pago del carrito

Objetivo: Resolver juntos un pago en línea con cálculos, cupón y respuesta asíncrona del banco.

Columna 1 — Requisitos

- R1. calcularSubtotal(productos): recorre el carrito, desestructura precio y cantidad, convierte el precio y retorna el subtotal.
- R2. aplicarCupon(subtotal, cupon): si el cupón (sin importar mayúsculas) es válido, descuenta 10 %. Muestra que cuponIngresado no cambió.
- R3. Suma el envío y muestra el total.
- R4. Completa procesarPago: si el total cabe en el saldo → callback(null, "aprobado"); si no → callback("Saldo insuficiente", null).
- R5. Callback mostrarRecibo: crea el recibo con ...cliente y muéstralo.
- R6. Justo después de pagar, imprime "Puedes seguir viendo productos...".

Salida esperada:

```
Subtotal: $134000
Cupón aplicado: -$13400
Cupón escrito por el cliente (sin cambios): promo10
Total a pagar: $128600
Procesando pago...
Puedes seguir viendo productos mientras confirmamos tu pago
Pago aprobado ← 1,5 s después
Recibo para Laura Gómez (laura@correo.com): $128600 — aprobado
```

Columna 2 — Código base

```
const cliente = {
 nombre: "Laura Gómez",
 correo: "laura@correo.com"
};
const carrito = [
 { nombre: "Audífonos", precio: 85000, cantidad: 1 },
 { nombre: "Cable USB", precio: "12000", cantidad: 2 },
 { nombre: "Funda", precio: 25000, cantidad: 1 },
];
const costoEnvio = "8000";
const cuponIngresado = "promo10";
const CUPON_VALIDO = "PROMO10";
const saldoTarjeta = 150000;
function procesarPago(total, callback) {
 console.log("Procesando pago...");
 setTimeout(function () {
 // R4
 }, 1500);
}
```

🏫 Guiado 2 · Registro de notas definitivas

Objetivo: Resolver juntos el cálculo de definitivas con datos que llegan de forma asíncrona.

Columna 1 — Requisitos

- R1. calcularPromedio(notas): convierte cada nota a número; si no hay notas, retorna null (evita NaN).
- R2. obtenerEstado(promedio): "Sin notas", "Aprobó" (3.0 o más) o "Reprobó".
- R3. Callback procesarNotas(registros): desestructura cada registro, crea un objeto nuevo con ...registro, definitiva y estado, e imprime la definitiva con un decimal.
- R4. Llama cargarNotas(procesarNotas) y luego imprime "Mientras tanto, revisa tu horario de clases".
- R5. Muestra cuántos aprobaron y comprueba que el registro original no cambió.

Salida esperada:

```
Consultando la plataforma académica...
Mientras tanto, revisa tu horario de clases
Camila: 4.1 (Aprobó) ← 2 s después
Julián: 3.0 (Aprobó)
Sofía: — (Sin notas)
Mateo: 2.6 (Reprobó)
Aprobaron 2 de 4
¿El registro original cambió? false
```

Columna 2 — Código base

```
const notasServidor = [
 { estudiante: "Camila", notas: [4.5, 3.8, 4.0] },
 { estudiante: "Julián", notas: [2.5, 3.0, "3.5"] },
 { estudiante: "Sofía", notas: [] },
 { estudiante: "Mateo", notas: [2.0, 2.8, 3.1] },
];
const NOTA_MINIMA = 3.0;
function cargarNotas(callback) {
 console.log("Consultando la plataforma académica...");
 setTimeout(function () {
 callback(notasServidor);
 }, 2000);
}
```

🏠 Para la casa

Objetivo: Practicar de forma autónoma con tres casos reales (enunciados completos en el README).

Ejercicio	Caso	Temas que combina
1	**Turnos de un consultorio**: llamar primero a pacientes prioritarios	setTimeout, callbacks, bucles, condicionales, operadores lógicos, conversión de tipos
2	**Cajero automático**: validar PIN y procesar retiros	callback (error, resultado), setTimeout, === con tipos distintos, %, spread
3	**Seguimiento de un paquete**: aplanar una pirámide de estados	Callback Hell, funciones con nombre, desestructuración, spread, "2.5" + 1

🔥 Reto integrador · Restaurante El Fogón (1/2)

Objetivo: Construir un programa completo de pedidos a domicilio que integre los temas de la clase.

Situación: el restaurante recibe un pedido, calcula el valor, confirma el pago y hace seguimiento hasta la entrega.

Req.	Qué debe hacer	Concepto que evalúa
R1	buscarPlato(codigo) sin importar mayúsculas; retorna el plato o null	Funciones, bucles, ===, métodos de cadena
R2	calcularSubtotal(items, avisar): avisa platos inexistentes o agotados y retorna { subtotal, lineas }	Callback síncrono, condicionales, conversión
R3	Total = subtotal + domicilio	Conversión de tipos
R4	confirmarPago(total, callback) en 1 s: error si supera el cupo	setTimeout, callback (error, resultado)
R5	Seguimiento: preparación → despacho → entrega, cada 1 s, sin pirámide	Callback Hell aplanado
R6	Factura con ...pedido; el pedido original no cambia	Spread y desestructuración
R7	"Puedes seguir navegando en la app" sale antes del pago	Orden de ejecución
R8	Responde en comentarios: ¿por qué "p2" fallaba? ¿por qué R7 sale antes?	Explicación

🔥 Reto integrador · Restaurante El Fogón (2/2)

Objetivo: Conocer el código base, el entregable y la rúbrica del reto.

Columna 1 — Entregable y rúbrica

Entregable: carpeta reto-apellido-nombre comprimida (.zip) con index.html, js/main.js comentado por requisito y una captura de la consola.

Rúbrica:

- R1 buscarPlato — 10 %
- R2 subtotal con callback — 20 %
- R3 total con conversión — 10 %
- R4 pago asíncrono — 15 %
- R5 seguimiento sin pirámide — 15 %
- R6 factura con spread — 10 %
- R7 orden de ejecución — 5 %
- R8 reflexión — 5 %
- Calidad del código (reglas de juego) — 10 %

Columna 2 — Código base

```
const menu = [
  { codigo: "P1", plato: "Bandeja paisa", precio: 32000, disponible:
true },
  { codigo: "P2", plato: "Ajiaco", precio: "28000", disponible: true },
  { codigo: "P3", plato: "Sancocho", precio: 26000, disponible: false
},
];
const pedido = {
  cliente: "Andrés Mora",
  direccion: "Calle 10 # 5-20",
  items: [
    { codigo: "P1", cantidad: 2 },
    { codigo: "p2", cantidad: 1 },
    { codigo: "P3", cantidad: 1 },
    { codigo: "P9", cantidad: 1 }
  ],
};
const DOMICILIO = "5000";
const CUPO_MAXIMO = 100000;
function esperar(mensaje, milisegundos, siguiente) {
  setTimeout(function () {
    console.log(mensaje);
    siguiente();
  }, milisegundos);
}
```

Total esperado: **$97000**, pago aprobado.

Resumen y antes de entregar

Objetivo: Cerrar la clase con las ideas clave y una lista de verificación.

Resumen:

Asincronía

El programa no espera a que terminen las tareas lentas.

No bloqueante

El hilo principal sigue; los callbacks llegan por la cola y el Event Loop.

Callbacks

Funciones pasadas como argumento; útiles, pero anidadas forman el Callback Hell.

Promesas y async/await

Formas modernas y más legibles de manejar la asincronía.

Antes de entregar:

- El <script> está justo antes de </body> y la consola no muestra errores.
- Cada requisito tiene su comentario // R1, // R2...
- Uso const por defecto, let solo si reasigno, y nunca var.
- Comparo con === / !== y convierto con Number() los textos numéricos.
- Paso callbacks sin paréntesis: funcion, no funcion().
- Revisé que el orden de la consola coincida con la salida esperada.